

ORCA Marine Intelligence Platform

Architecture & Implementation Plan | SIH 2026 (Problem 26178)

1. Data Collection & API Strategy

The system relies on fetching real-time and archival marine data across three core government endpoints:

- **INCOIS (ERDDAP Server):** Utilize HTTP REST endpoints requesting JSON/CSV payloads for immediate access to Potential Fishing Zones (PFZ) and daily Ocean State Forecasts (wave heights, swells).
- **MOSDAC (Data Pipeline):** Integrate Python libraries (`requests`, `netCDF4`) to query catalog metadata and parse INSAT-3D HDF5/NetCDF files for spatial chlorophyll mapping and cyclone storm tracks.
- **Bhuvan (Web Services):** Consume Bhuvan's REST APIs for dynamic reverse-geocoding, and load OGC-compliant WMS (Web Map Service) endpoints for rendering maritime boundaries directly onto the user interface.

2. AI Model Strategy

A hybrid modeling approach maximizes speed during development while ensuring stability in production:

- **Development Environment:** The M4 Pro chip with 48GB of unified RAM provides enormous headroom to run high-parameter open-weights models (like Llama 3 8B or Qwen) completely locally. This allows for zero-latency testing of agent tool-calling loops without hitting API rate limits.
- **Production Environment:** Transition to Google AI Studio to leverage **Gemini 1.5 Flash**. It is highly optimized for fast function-calling and naturally supports Indian regional languages for the conversational interface.
- **Constraint:** The model will be strictly constrained using rigid System Prompts to act exclusively as a Marine Intelligence Orchestrator.

3. Platform Architecture & Hosting

The platform separates the user interface from the heavy GIS and AI processing:

- **Frontend:** A responsive web application built using HTML and JavaScript. Firebase will be integrated to handle user session states and secure authentication seamlessly.
- **Backend:** A robust Python API built with FastAPI. This server will orchestrate the LLM, execute the GIS spatial logic, and handle NetCDF scientific data parsing.
- **Deployment:** The frontend can be hosted on platforms like Vercel, while the backend API requires a cloud environment (like Render or Koyeb) capable of running Python environments with geospatial dependencies.

4. Multi-Agent Orchestration Framework

A centralized reasoning engine (using LangGraph or CrewAI) will delegate tasks to specialized agents:

1. Orchestrator Agent

Analyzes the incoming natural language query, detects the regional language (e.g., Hindi, Tamil), and extracts the core intent to route to the correct specialist agent.

2. Ocean Analytics Agent

Executes API calls to INCOIS to retrieve the day's precise PFZ coordinates and localized wave conditions.

3. Weather Intelligence Agent

Scrapes and analyzes MOSDAC atmospheric data to verify if conditions (cyclones, lightning) permit safe maritime travel.

4. Geospatial Risk Agent

Performs spatial intersection checks, comparing the user's GPS coordinates against Bhuvan's Exclusive Economic Zone (EEZ) vector shapes to trigger geofencing alerts.

5. Synthesis & Safety Agent

Compiles the findings, resolves any conflicting data, formats the final response into the user's original language, and packages the map rendering coordinates for the frontend interface.